EL-2003 – Computer Organization & Assembly Language Lab
Lab # 8

Objectives:

1. Explain how the stack works (last in, first out, growing downwards) and predict SP after each push, pop, call and ret.
2. Use push and pop to save and restore registers, reverse values and swap registers.
3. Write subroutines with call and ret and place them correctly in the program.
4. Pass parameters to a subroutine through registers and through the stack, reading them with BP and removing them with ret n.
5. Preserve the caller's registers and return answers in AX.
6. Use the stack as a data structure to reverse a string, and follow nested calls.
7. Verify results in AFD by watching SP, the Stack list and the memory dump.

Note: Carefully read the following instructions (*Each instruction contains a weightage*)

1. First think about statement problems and then write your program.
2. Do not copy from any source otherwise you will be penalized with negative marks.
3. Add your source code in word document and attach output screenshots.
4. Upload word file with codes and screenshot.
5. Submit your lab on Google Classroom.

Question 1:

Write square, which takes a number in AX and returns its square in AX without changing DX. Its very first instruction stores SP in sp_deep. Write sumsq, which takes one number in BX and another in CX and returns $BX^2 + CX^2$ in AX. It calls square twice and must give back BX and CX unchanged. It saves exactly those two registers and nothing else. Call sumsq first with BX=3 and CX=4, storing the answer in r1. Then call it with BX=5 and CX=12, storing the answer in r2. After the second call, store BX in bx_a and CX in cx_a. Store SP before the first call and after the last call in sp_in and sp_out.

Question 2:

Write square, which takes a number in AX and returns its square in AX. Its very first instruction stores SP in sp_deep. Write sumsqarr, which takes an array address in BX and a count in CX and returns the sum of the squares of all elements in AX. It calls square once for every element. It must save and restore exactly BX, CX, DX and SI, in that order of pushing, and pop them in reverse. Run it on dw 2, 3, 4, 5 with a count of 4, store the answer in total, and store BX and CX after the call in bx_a and cx_a. Store SP before and after the call in sp_in and sp_out.

Question 3:

A house has three rooms stored as pairs of words, length then width: dw 5,4, 6,3, 7,2. Carpet costs 25 rupees per square unit. Write a subroutine area that returns length × width in AX, and a subroutine total_area that goes through all the rooms, uses area for each one, and returns the total area in AX. You decide how area receives its two numbers and which registers each subroutine must protect. Back in the main code, multiply the total area by the rate to get the cost. Store the total area in tarea and the cost in cost, and make sure SP is the same before and after the work.